

BÁO CÁO BÀI TẬP LỚN: HỆ THỐNG ĐẤU GIÁ TRỰC TUYẾN AUREX AUCTION

Môn học: Lập trình nâng cao Dự án: Aurex Auction System Nhóm thực hiện: Team 6

I. GIỚI THIỆU MỤC TIÊU VÀ PHẠM VI THỰC HIỆN

1. Mục tiêu dự án

Dự án được xây dựng với mục tiêu thiết lập một nền tảng đấu giá trực tuyến hiện đại, minh bạch và có khả năng tương tác thời gian thực cao. Nhóm chúng em tập trung giải quyết các bài toán về xử lý đồng thời, truyền tải dữ liệu qua mạng an toàn và tối ưu hóa trải nghiệm người dùng trên môi trường phân tán.

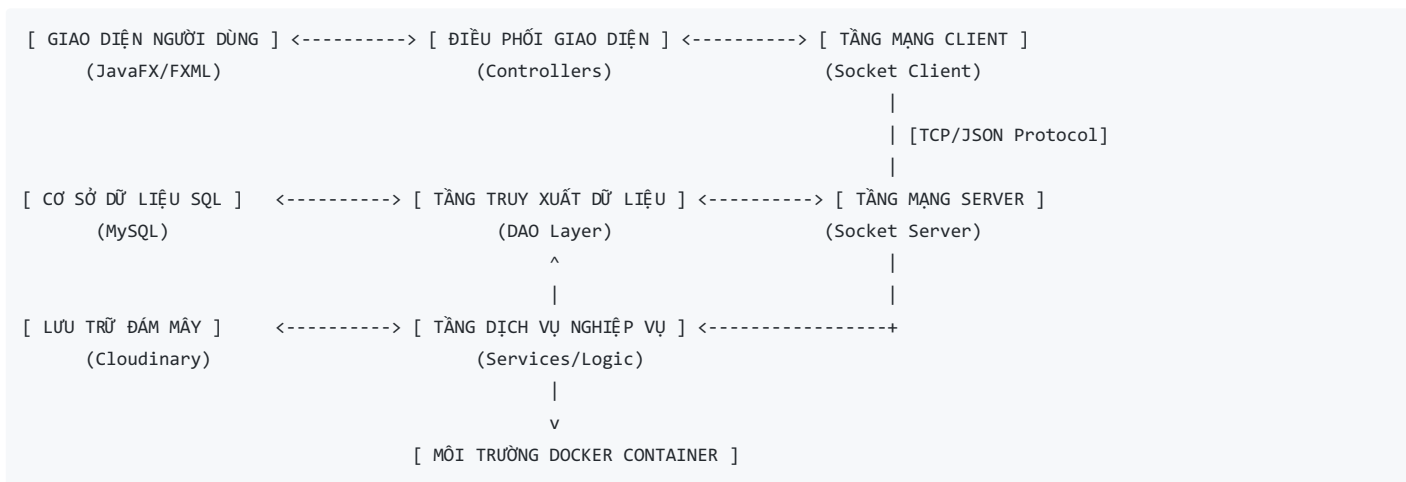
2. Phạm vi thực hiện

Hệ thống bao quát toàn bộ quy trình nghiệp vụ từ quản lý tài khoản, đăng bán tài sản, tham gia đấu giá trực tiếp đến khâu kết thúc và xử lý hậu đấu giá. Chúng em đã áp dụng kiến trúc Client-Server với giao thức TCP tùy chỉnh, kết hợp với các công nghệ lưu trữ đám mây và ảo hóa (Docker) để đảm bảo hệ thống có khả năng triển khai linh hoạt.

II. KIẾN TRÚC TỔNG THỂ HỆ THỐNG

Hệ thống được thiết kế theo mô hình phân tầng (Layered Architecture) kết hợp với kiến trúc micro-service nhẹ thông qua việc đóng gói các thành phần.

1. Sơ đồ kiến trúc tổng quát



2. Mô tả các thành phần kiến trúc

- Tầng Client:** Sử dụng JavaFX để xây dựng giao diện người dùng. Tầng này chịu trách nhiệm hiển thị thông tin và tương tác trực tiếp với người dùng cuối, đảm bảo giao diện luôn mượt mà nhờ cơ chế xử lý bất đồng bộ.
- Giao thức Truyền thông:** Chúng em xây dựng một giao thức riêng trên nền TCP, đóng gói dữ liệu dưới dạng JSON. Điều này giúp giảm thiểu độ trễ so với giao thức HTTP truyền thống, đáp ứng yêu cầu khắt khe về thời gian thực của các phiên đấu giá.
- Tầng Server:** Được thiết kế để chạy độc lập và có thể đóng gói thành Docker Image. Server sử dụng `CachedThreadPool` để quản lý các luồng (thread) kết nối, giúp hệ thống không bị treo khi có hàng ngàn yêu cầu cùng lúc.
- Tầng Ảo hóa (Docker):** Toàn bộ môi trường chạy Server, Database và các cấu hình hệ thống được đóng gói thông qua Docker và Docker Compose, giúp việc triển khai dự án trở nên nhất quán trên mọi máy tính.

III. CHI TIẾT CÁC CHỨC NĂNG ĐẠT ĐƯỢC

1. Quản lý vai trò Người dùng (Sự hợp lý trong thiết kế)

Trong hệ thống Aurex Auction, thay vì tách biệt rạch ròi hai vai trò **Bidder** (Người mua) và **Seller** (Người bán) thành các tài khoản riêng biệt, chúng em đã tích hợp cả hai khả năng này vào một thực thể duy nhất là **User**.

Tính hợp lý của giải pháp:

- Thực tế nghiệp vụ:** Một người dùng trong thực tế hoàn toàn có nhu cầu vừa đăng bán sản phẩm cá nhân, vừa tham gia đấu giá các sản phẩm của người khác.

- Việc gộp vai trò giúp người dùng không phải đăng ký và quản lý nhiều tài khoản.
- Thống nhất tài chính:** Người dùng có một ví tiền duy nhất (Balance). Tiền kiếm được từ việc bán đấu giá tài sản có thể được dùng ngay lập tức để đặt giá cho các tài sản khác, tạo nên dòng tiền luân chuyển thuận tiện.
 - Tối ưu cấu trúc OOP:** Chúng em sử dụng tính kế thừa để xây dựng lớp `User` cơ bản chứa đầy đủ các thuộc tính của cả hai vai trò, giúp giảm thiểu sự phức tạp khi truy vấn dữ liệu và quản lý phiên đăng nhập.

2. Nhóm chức năng bắt buộc

- Xác thực và Bảo mật:** Mật khẩu được mã hóa qua thuật toán **BCrypt** (mã hóa một chiều mạnh nhất hiện nay). Ngoài ra, hệ thống còn hỗ trợ xác thực **OTP** qua Email cho tính năng quên mật khẩu.
- Quản lý phiên đấu giá:** Tự động chuyển đổi trạng thái phiên dựa trên thời gian thực: `OPEN` -> `RUNNING` -> `FINISHED`.
- Đấu giá trực tiếp:** Hệ thống kiểm tra điều kiện đấu giá nghiêm ngặt, chỉ chấp nhận mức giá cao hơn giá hiện tại cộng với bước giá tối thiểu.

3. Nhóm chức năng nâng cao (Điểm cộng)

- Xử lý đấu giá đồng thời (Concurrency Handling):** Áp dụng kỹ thuật **Pessimistic Locking** (`SELECT ... FOR UPDATE`) để khóa hàng dữ liệu trong MySQL khi có giao dịch. Điều này đảm bảo không có hai người dùng cùng thắng hoặc giá bị sai lệch khi đặt giá cùng lúc.
- Thuật toán Anti-sniping:** Nếu có lượt đặt giá trong 2 phút cuối, hệ thống tự động gia hạn thêm 2 phút. Điều này công bằng cho mọi người tham gia và tối đa hóa lợi nhuận cho người bán.
- Triển khai với Docker & Containerization:**
 - Hướng giải quyết:* Xây dựng `Dockerfile` đa tầng (Multi-stage build) để nén dung lượng ảnh và sử dụng `docker-compose` để điều phối Server và MySQL.
 - Lý do:* Giúp hệ thống có khả năng mở rộng (Scaling) dễ dàng và loại bỏ hoàn toàn lỗi "chạy được trên máy em nhưng không chạy được trên máy thầy".
- Realtime Price Curve:** Biểu đồ đường biến động giá được cập nhật tức thời mà không cần refresh, mang lại trải nghiệm chuyên nghiệp như các sàn chứng khoán.

IV. PHÂN CHIA CÔNG VIỆC THÀNH VIÊN (TEAM 6)

Nhóm chúng em phân chia công việc dựa trên chuyên môn để tối ưu hóa hiệu suất:

Nhóm chuyên trách	Thành viên	Trách nhiệm chính
Network & Logic	Hoàng Trung Dũng Nguyễn Tấn Dũng	- Phát triển Server TCP, thiết lập môi trường Docker. - Xây dựng Database, xử lý giao dịch SQL và Concurrency. - Hiện thực hóa thuật toán Anti-sniping, Auto-bid và Security.
UI & Controller	Trần Văn Thuyết Phạm Khắc Quốc Trung	- Thiết kế giao diện JavaFX/FXML và styling CSS. - Xử lý luồng dữ liệu tại Client và điều phối Scene. - Tích hợp đồ thị thời gian thực và quản lý State người dùng.

V. CÔNG NGHỆ VÀ KỸ THUẬT SỬ DỤNG

- Ngôn ngữ:** Java 17 (LTS).
- Công nghệ mạng:** Java Socket (TCP), JSON (Gson).
- Giao diện:** JavaFX 21, LineChart API.
- Cơ sở dữ liệu:** MySQL 8.0, JDBC.
- Triển khai:** Docker, Docker Compose.
- Bảo mật:** BCrypt, JavaMail (OTP).

VI. KẾT LUẬN

Báo cáo trên đây đã tóm lược toàn bộ quá trình nghiên cứu và thực hiện bài tập lớn của Team 6. Chúng em đã nỗ lực vận dụng những kiến thức nâng cao về lập trình hướng đối tượng, xử lý mạng và ảo hóa để tạo nên một sản phẩm hoàn thiện. Hệ thống không chỉ đáp ứng các yêu cầu về chức năng mà còn đảm bảo tính ổn định và khả năng triển khai thực tế cao.